

# Virtualización en Linux

Conceptos, tipos y herramientas

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV · Infraestructura Virtual

IV · UNIDAD 1 — VIRTUALIZACIÓN EN LINUX

# 01

## **Introducción a la virtualización**

---

Concepto, usos, ventajas y conceptos básicos

## ¿Qué es la virtualización?

*" La **virtualización** utiliza el software para imitar las características del hardware y crear un sistema informático virtual.*

### IDEA FUNDAMENTAL

- Permite ejecutar **varios sistemas virtuales** sobre un mismo equipo físico
- Múltiples **sistemas operativos** y aplicaciones funcionando en paralelo
- Aumenta el **rendimiento** del hardware disponible
- Aprovecha el tiempo de procesamiento que normalmente se desperdicia

 La virtualización es la base sobre la que se construyen las modernas plataformas **cloud** y los sistemas de despliegue automatizado.

## ¿Para qué se utiliza?

### PRODUCCIÓN Y SERVICIOS

- **Aislamiento e independencia** de servicios y contenidos
- **Migración en vivo** entre servidores
- Creación de **clústeres** y sistemas distribuidos

### DESARROLLO Y FORMACIÓN

- **Laboratorio de pruebas** sin riesgo
- Virtualización de **arquitecturas** de las que no se dispone
- **Herramientas de aprendizaje** y experimentación

## Ventajas y desventajas

### VENTAJAS

- Importante **ahorro económico**
- Mayor **seguridad** y aislamiento
- Mejor **aprovechamiento** de recursos
- **Migración en vivo** entre hosts
- Notable **ahorro energético**

### DESVENTAJAS

- Muchos sistemas dependen de un **único equipo físico**
- Ligera **penalización** en rendimiento
- Mayor **complejidad** de gestión



La **alta disponibilidad** es imprescindible cuando concentramos varios servicios virtualizados en un mismo host.

## Conceptos básicos

### ANFITRIÓN (*HOST*)

Sistema operativo que ejecuta el software de virtualización. **Controla el hardware real.**

### INVITADO / HUÉSPED (*GUEST*)

Sistema operativo **virtualizado** que se ejecuta sobre el anfitrión.

### HIPERVISOR

Software de virtualización que **gestiona los invitados** y reparte los recursos del host.

### SOPORTE HARDWARE

Desde 2005, **Intel VT** y **AMD-V** añaden extensiones de virtualización al procesador para mejorar el rendimiento.

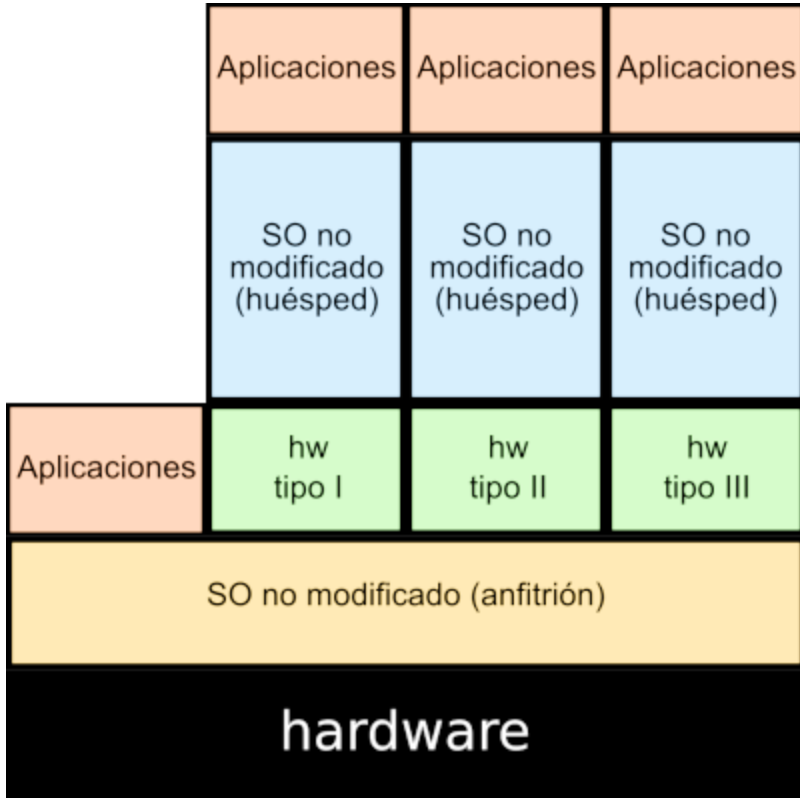
# 02

## Tipos de virtualización

---

Emulación, hardware, completa, paravirtualización y contenedores

# Emulación



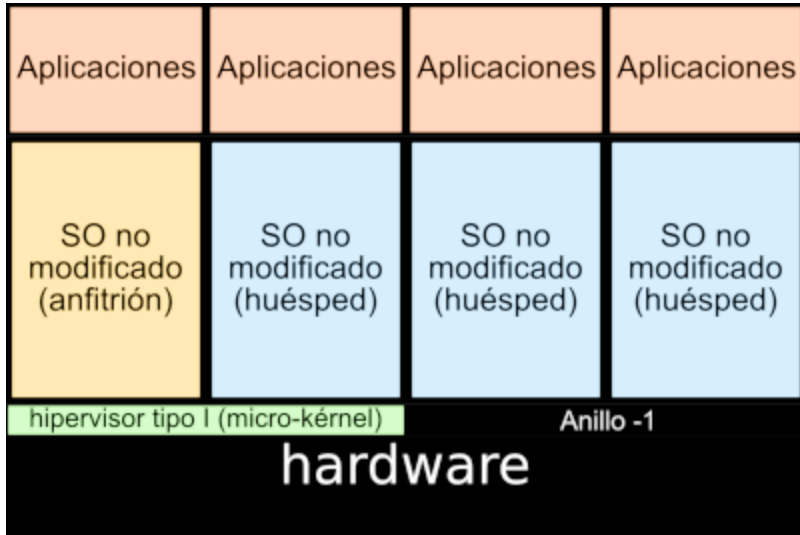
- El hipervisor **imita por software** una arquitectura completa: procesador, memoria, instrucciones, comunicaciones...
- Los programas se ejecutan creyendo que están sobre una **arquitectura concreta**
- Útil para correr software de **otra plataforma** (ej: ARM sobre x86)
- **Rendimiento bastante bajo**

## EJEMPLOS

QEMU · Microsoft Virtual PC · Wine



## Virtualización por hardware

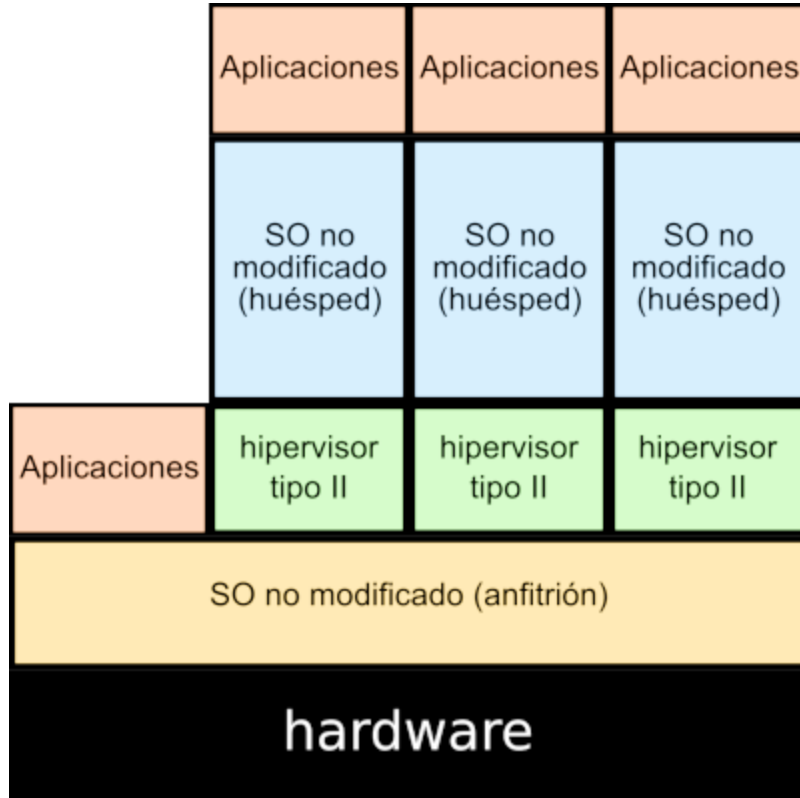


- El hipervisor simula **suficiente hardware** para que un sistema operativo **no adaptado** se ejecute aislado
- Se usan **hipervisores de tipo 1**, que controlan directamente el hardware físico del host
- La CPU **debe disponer** de las extensiones de virtualización (Intel VT / AMD-V)
- Es la opción de **mayor rendimiento** entre las virtualizaciones completas

### EJEMPLOS

Xen · KVM · Microsoft Hyper-V · VMware ESXi

## Virtualización completa

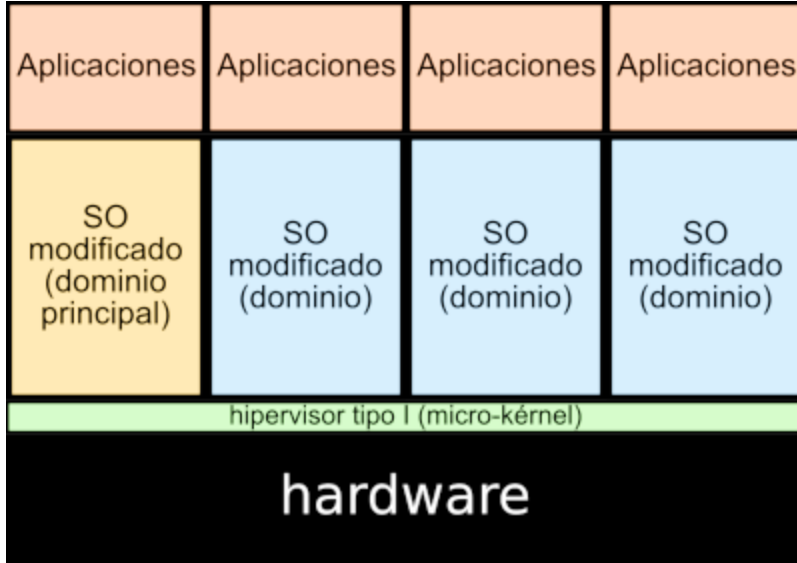


- También permite ejecutar **SO no adaptados** en aislamiento
- Se usan **hipervisores de tipo 2**, instalados sobre el sistema operativo del host
- **No controlan** directamente el hardware físico
- Ofrecen **menos rendimiento** que la virtualización por hardware

### EJEMPLOS

VirtualBox · VMware Workstation · VMware Player · Parallels Desktop

## Paravirtualización

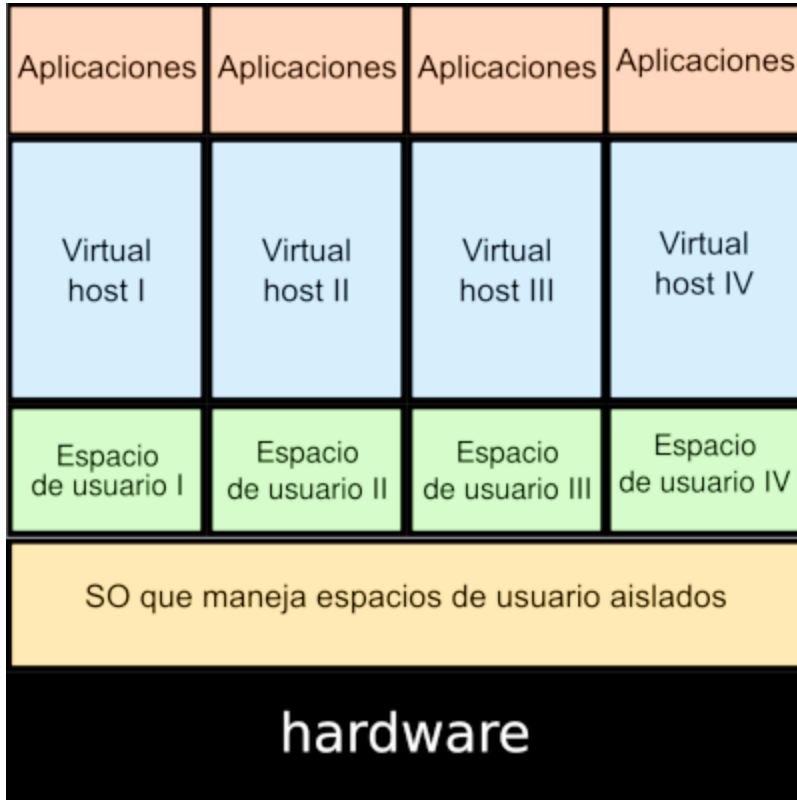


- El hipervisor ofrece una **interfaz especial** para acceder a los recursos
- A veces requiere **modificar el SO** de la máquina virtual
- Ofrecen el **máximo rendimiento**
- Pero **no se pueden usar** sistemas operativos sin modificar ni hardware específico

### EJEMPLOS

Xen · Microsoft Hyper-V · VMware ESXi

## Virtualización ligera o en contenedores



- También llamada **virtualización a nivel de SO**
- Sobre el **núcleo** del SO se ejecuta una capa que permite múltiples **espacios de usuario aislados** (contenedores)
- Un contenedor es un **conjunto de procesos** aislados con:
  - **Sistema de ficheros** propio
  - **Configuración de red** propia
  - Acceso a recursos del host (CPU, memoria)

## Tipos de contenedores

### CONTENEDORES DE SISTEMA

Su uso es similar al de una máquina virtual tradicional:

- Se accede por **SSH**
- Se **instalan servicios**, se actualizan
- Ejecutan un **conjunto de procesos**

#### Ejemplo

LXC (*Linux Container*)

### CONTENEDORES DE APLICACIÓN

Pensados para el **despliegue** de aplicaciones (especialmente web):

- Una aplicación = un contenedor
- Imágenes versionadas y portables
- Base del *cloud-native*

#### Ejemplos

Docker · Podman

# Resumen comparativo

| TIPO                    | HIPERVISOR | SO MODIFICADO | RENDIMIENTO | EJEMPLOS           |
|-------------------------|------------|---------------|-------------|--------------------|
| Emulación               | —          | No            | Muy bajo    | QEMU, Wine         |
| Virtualización HW       | Tipo 1     | No            | Alto        | KVM, Xen, ESXi     |
| Virtualización completa | Tipo 2     | No            | Medio       | VirtualBox, VMware |
| Paravirtualización      | Tipo 1     | Sí            | Muy alto    | Xen, Hyper-V       |
| Contenedores            | —          | —             | Casi nativo | LXC, Docker        |

# 03

## **QEMU/KVM y libvirt**

---

Virtualización completa en Linux

# QEMU

" ***QEMU** es un emulador genérico y de código abierto de máquinas virtuales.*

## DOS MODOS DE FUNCIONAMIENTO

### MODO EMULADOR

- Permite ejecutar SO de una **arquitectura distinta** (ej. ARM sobre x86)
- Útil para **desarrollo cruzado**
- Rendimiento bajo

### MODO VIRTUALIZACIÓN

- Apoyado en hipervisores como **KVM**
- Aprovecha las extensiones del procesador
- **Alto rendimiento**



# KVM

*" Kernel-based Virtual Machine es un hipervisor de **tipo 1** integrado al kernel de Linux.*

## CARACTERÍSTICAS

- Solución de **virtualización completa** para Linux
- Necesita CPU con extensiones **Intel VT** o **AMD-V**
- Se compone de varios **módulos del kernel**:
  - `kvm.ko` — infraestructura base de virtualización
  - `kvm-intel.ko` / `kvm-amd.ko` — módulo específico del procesador

 **QEMU + KVM** es la combinación habitual: QEMU emula los dispositivos y KVM acelera la ejecución del invitado.

## Dispositivos paravirtualizados ( `virtIO` )

- En virtualización completa, los dispositivos (discos, red...) están **emulados por software**
- La VM interactúa con ellos como si fuesen físicos → **poco rendimiento**
- KVM ofrece una alternativa: los **dispositivos paravirtualizados**, agrupados como `virtIO`

### DISPOSITIVOS EMULADOS

- Compatibilidad universal
- **Rendimiento bajo**
- Cada operación atraviesa la capa de emulación

### DISPOSITIVOS `VIRTIO`

- Drivers específicos en el invitado
- **Rendimiento muy cercano al real**
- Recomendado para discos y tarjetas de red

# libvirt

" *libvirt es la API y conjunto de herramientas que facilita la **gestión** de los recursos virtualizados.*

## ¿POR QUÉ LIBVIRT?

- Trabajar directamente con QEMU/KVM es **complejo**
- libvirt ofrece una **API genérica** y un **demonio** comunes
- Soporta varios sistemas: **KVM, LXC, Xen...**
- Permite usar las **mismas herramientas** independientemente del hipervisor

## Mecanismos de conexión a libvirt

### LOCAL SIN PRIVILEGIOS

```
qemu:///session
```

- Acceso a las VM **del usuario actual**
- Sin permisos para crear redes
- Útil para usuarios de escritorio

### LOCAL PRIVILEGIADO

```
qemu:///system
```

- Acceso a las VM **del sistema**
- Permisos completos sobre red y almacenamiento

### REMOTO PRIVILEGIADO POR SSH

```
qemu+ssh:///system
```

- Conexión **a un servidor remoto** que ejecuta libvirt
- Autenticación a través de **SSH**
- Base para administrar **clústeres** de hipervisores

## Aplicaciones del ecosistema libvirt

| APLICACIÓN          | FUNCIÓN                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>virsh</b>        | Cliente oficial de <b>línea de comandos</b> . Shell completa para la API                                                         |
| <b>virt-manager</b> | Aplicación <b>gráfica</b> con la mayor parte de las funcionalidades                                                              |
| <b>virt-install</b> | Creación de MV desde la <b>línea de comandos</b> ( <code>virt-install</code> , <code>virt-clone</code> , <code>virt-xml</code> ) |
| <b>virt-viewer</b>  | Acceso a la <b>consola gráfica</b> de una VM                                                                                     |
| <b>gnome-boxes</b>  | Aplicación gráfica <b>simple</b> para usuarios de escritorio                                                                     |



`virsh` es la herramienta de referencia para automatizar y administrar libvirt en servidores sin entorno gráfico.

IV · UNIDAD 1 — VIRTUALIZACIÓN EN LINUX

# 04

## Introducción a LXC

---

Contenedores de sistema en Linux

## Linux Containers (LXC)

" *LXC es una tecnología de **virtualización ligera** o por contenedores, mantenida por Canonical.*

### COMPONENTES DEL KERNEL QUE LA HACEN POSIBLE

#### GRUPOS DE CONTROL (*CGROUPS*)

- En Debian 11 se utiliza **cgroups v2**
- **Limitan el uso de recursos** (memoria, CPU, I/O, red) de un proceso y sus hijos

#### ESPACIOS DE NOMBRES (*NAMESPACES*)

- Proporcionan una **vista aislada** a un proceso
- Aíslan: **interfaces de red, procesos, usuarios**, ficheros...

# LXD

" **LXD** (*Linux Container Daemon*) es la herramienta de **gestión** de contenedores y máquinas virtuales para Linux, también desarrollada por Canonical.

## CARACTERÍSTICAS

- Ofrece una **REST API** que se puede consumir con la línea de comandos o herramientas de terceros
- Gestiona **instancias**, que pueden ser de dos tipos:
  - **Contenedores** — usando LXC internamente
  - **Máquinas virtuales** — usando QEMU internamente
- Una sola herramienta para administrar **ambos modelos**

✓ LXD unifica la gestión de contenedores de sistema y de máquinas virtuales bajo el **mismo flujo de trabajo**.



## ¿Cuándo elegir cada opción?

### KVM / LIBVIRT

Cuando se necesita un **sistema operativo completo** y aislado con el **mismo kernel** que en producción.

**Ideal para:** servidores tradicionales, laboratorios.

### LXC / LXD

Cuando se busca un **contenedor de sistema** ligero pero con experiencia similar a una VM.

**Ideal para:** múltiples servicios en un host, desarrollo, *self-hosting*.

### DOCKER / PODMAN

Cuando se quiere desplegar **una aplicación** empaquetada y portable.

**Ideal para:** microservicios, *cloud-native*, CI/CD.

IV · UNIDAD 1 — VIRTUALIZACIÓN EN LINUX

# ¡Gracias!

Virtualización en Linux

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV